

Projet C++ - Ultimate Tic-Tac-Toe

Récapitulatif technique des classes

ESILV - A3 Alternant

Mai 2026

1 Introduction

Ce document récapitule l'architecture logicielle mise en place pour le projet Ultimate Tic-Tac-Toe. L'objectif est de fournir une simulation interne robuste capable de s'interfacer avec la librairie graphique fournie, tout en permettant le développement d'IA performantes.

Le projet est structuré autour de ces fichiers principaux :

- `main.h` / librairie (`.lib`) : fournis avec le projet, ne sont jamais modifiés.
- `GameBoard.h` : définit toutes les structures de données du jeu.
- `GameRules.h` : implémente les règles du jeu.
- `MyIA.h` : contient l'algorithme de décision de l'IA.
- `main.cpp` : assure la synchronisation entre la librairie et le simulateur interne.

2 Structures et Énumérations de Base

Toutes les structures de données sont définies dans `GameBoard.h`.

2.1 Player (`GameBoard.h`)

Énumération définissant l'état d'une case ou l'identité d'un joueur.

- `NONE` : case vide, aucun joueur.
- `X` : symbole du premier joueur (croix).
- `O` : symbole du second joueur (ronds).

Choix technique : L'utilisation d'un `enum class` (plutôt que des entiers bruts) garantit la lisibilité et la sécurité de type — le compilateur interdit toute affectation accidentelle d'une valeur invalide.

2.2 Move (`GameBoard.h`)

Structure stockant les coordonnées complètes d'un coup dans le système interne à 4 coordonnées.

- `bigRow`, `bigCol` : indices (0-2) de la petite grille dans le plateau global.
- `smallRow`, `smallCol` : indices (0-2) de la case dans la petite grille choisie.

La structure fournit deux constructeurs et deux méthodes de conversion :

```
Move() // constructeur par défaut
- coup invalide (-1,-1,-1,-1)
Move(int br, int bc, int sr, int sc) // constructeur avec
  coordonnées réelles
bool isValid() const // retourne true si le
  coup a été initialisé
GameMove toGameMove() const // convertit vers le
  format plat (0-8) du prof
static Move fromGameMove(const GameMove& gm) // convertit
  depuis le format du prof
```

Formule de conversion : $\text{bigRow} = \text{row} / 3$, $\text{smallRow} = \text{row} \% 3$ (et identique pour les colonnes). Exemple : $\text{row}=5$, $\text{col}=7 \rightarrow \text{bigRow}=1$, $\text{bigCol}=2$, $\text{smallRow}=2$, $\text{smallCol}=1$.

2.3 SmallBoard (GameBoard.h)

Structure représentant une petite grille de Tic-Tac-Toe standard (3×3).

- `cells[3][3]` : tableau 2D de Player indiquant qui a joué dans chaque case.
- `isWon` : booléen indiquant si cette petite grille a été remportée.
- `winner` : le Player qui a gagné cette petite grille (NONE si pas encore gagnée).

Le constructeur initialise toutes les cases à `Player::NONE`, `isWon` à `false`, et `winner` à `Player::NONE`.

2.4 GameState (GameBoard.h)

Structure centrale du simulateur. Contient l'intégralité de l'état du jeu à un instant donné.

- `boards[3][3]` : les 9 petites grilles `SmallBoard` constituant le plateau complet.
- `bigBoard[3][3]` : tableau de Player indiquant qui a remporté chaque petite grille (utilisé pour détecter la victoire finale).
- `currentPlayer` : le Player dont c'est le tour de jouer.
- `isBoardForced` : booléen indiquant si le prochain joueur est contraint à une grille spécifique.
- `forcedRow`, `forcedCol` : indices de la grille forcée (-1 si libre de choisir).

Distinction clé : `boards[3][3]` contient les cases réelles (9 cellules chacune), tandis que `bigBoard[3][3]` ne contient qu'un Player par entrée — le vainqueur de chaque petite grille. C'est `bigBoard` qui détermine le vainqueur final.

3 Logique du Jeu (GameRules.h)

La classe `GameRules` regroupe toutes les règles du jeu sous forme de méthodes statiques. Elle ne contient aucune donnée propre — c'est une boîte à outils pure. Toutes les méthodes sont `static`, ce qui signifie qu'elles s'appellent directement sur la classe sans créer d'instance :

```
GameRules::playMove(state, move);  
GameRules::getLegalMoves(state);
```

3.1 checkSmallWin(board, player)

Vérifie si un joueur a aligné 3 symboles identiques dans une petite grille 3×3. Analyse les 8 combinaisons gagnantes possibles : 3 lignes, 3 colonnes, 2 diagonales. Retourne true dès qu'une combinaison gagnante est trouvée.

3.2 checkBigWin(bigBoard, player)

Applique la même logique que checkSmallWin, mais sur le plateau global bigBoard[3][3]. Détermine si un joueur a aligné 3 petites grilles remportées. C'est cette fonction qui détecte la victoire finale de la partie.

3.3 getWinner(state)

Interroge checkBigWin pour X puis pour O. Retourne le Player vainqueur, ou Player::NONE si la partie n'est pas encore terminée.

3.4 isSmallBoardPlayable(board)

Vérifie qu'une petite grille peut encore recevoir des coups : elle ne doit pas être gagnée (isWon == false) et doit contenir au moins une case vide (Player::NONE).

3.5 getLegalMoves(state)

Génère la liste de tous les coups légaux à partir de l'état courant. Retourne un std::vector<Move>.

- Si isBoardForced == true : seuls les coups dans la grille forcée (forcedRow, forcedCol) sont ajoutés.
- Si isBoardForced == false : les coups libres dans toutes les grilles encore jouables sont ajoutés.

3.6 playMove(state, move)

Applique un coup sur le GameState et met à jour l'ensemble des informations dans l'ordre suivant :

- Place le symbole du currentPlayer dans la case ciblée.
- Vérifie si la petite grille est désormais gagnée via checkSmallWin — si oui, met à jour isWon, winner, et bigBoard.
- Calcule la prochaine grille forcée : la position de la case jouée (smallRow, smallCol) devient les indices de la prochaine grille. Si cette grille n'est pas jouable, isBoardForced passe à false (coup libre).

- Alterne le currentPlayer : X devient O, O devient X.

3.7 isGameOver(state)

Retourne true si la partie est terminée, pour deux raisons possibles : un joueur a gagné le plateau global (getWinner != NONE), ou il ne reste plus aucun coup légal (getLegalMoves retourne une liste vide — match nul).

4 Intelligence Artificielle (MyIA.h)

La classe MyIA contient l'algorithme de décision. Elle expose une seule méthode publique, getMove(), et organise ses méthodes internes en sections public et private.

4.1 Structure de la classe

La méthode publique getMove() est le point d'entrée unique appelé depuis main.cpp. Les méthodes privées sont des outils internes non accessibles depuis l'extérieur.

Méthode	Visibilité	Rôle
getMove(state)	public	Point d'entrée — retourne le meilleur coup à jouer
minimax(...)	private	Calcule le score d'une position par simulation récursive
evaluate(state, me)	private	Estime un score quand minimax atteint la profondeur 0

4.2 Première implémentation : IA aléatoire

La première version de getMove() est une IA aléatoire. Elle sert à valider que le moteur de jeu fonctionne correctement avant d'implémenter une vraie stratégie.

Fonctionnement :

- Récupère tous les coups légaux via GameRules::getLegalMoves(state).
- Si la liste est vide, retourne un coup invalide Move() — protection contre un crash.
- Sélectionne un coup au hasard via rand() % legalMoves.size().
- Retourne ce coup.

Pourquoi rand() % size() donne toujours un coup valide : getLegalMoves() filtre déjà tous les coups illégaux — la liste ne contient que des coups jouables. L'index aléatoire est donc forcément dans les bornes.

5 Intégration (main.cpp)

Le fichier main.cpp assure la synchronisation entre la librairie libUTTTLib fournie et le simulateur interne. Il remplace le template fourni avec le projet.

5.1 Interface fournie avec le projet

La librairie expose l'objet game de type IGame& (déclaré extern dans main.h). Les méthodes disponibles sont :

- game.initialize(nbGame, level, mode, alwaysFirst, pseudo) : démarre une session.
- game.isAllGameFinish() : retourne true quand toutes les parties sont terminées.
- game.isFinish() : retourne true quand la partie en cours est terminée.
- game.getMove(gameMove) : remplit gameMove avec le dernier coup adverse en coordonnées plates (row/col de 0 à 8).
- game.setMove(gameMove) : envoie notre coup au moteur en coordonnées plates.
- game.getWinner() : retourne le vainqueur de la partie terminée.

5.2 Flux de jeu à chaque tour

À chaque tour, main.cpp effectue exactement trois opérations dans cet ordre :

Étape	Appel	Description
1	game.getMove(iaGameMove)	Récupère le coup adverse. Si row != -1, convertit via Move::fromGameMove() et appelle GameRules::playMove() pour mettre à jour le GameState interne.
2	MyIA::getMove(state)	L'IA analyse le GameState et retourne le meilleur coup dans le format interne (4 coordonnées).
3	game.setMove(myGameMove)	Convertit notre coup en GameMove (format plat 0-8) via toGameMove() et l'envoie au moteur.

5.3 Gestion des parties multiples

La boucle externe while (!game.isAllGameFinish()) couvre l'ensemble des parties de la session. À chaque nouvelle partie, un GameState est réinitialisé (constructeur appelé automatiquement). La boucle interne while (!game.isFinish()) gère les tours d'une partie individuelle.

6 Règles du Jeu Implémentées

Les règles suivantes sont strictement respectées conformément au cahier des charges :

- Le premier joueur peut jouer dans n'importe quelle grille au premier coup (`isBoardForced == false` au démarrage).
- Chaque coup détermine la grille forcée du joueur suivant : la position (`smallRow`, `smallCol`) du coup joué correspond aux indices (`bigRow`, `bigCol`) de la prochaine grille.
- Si la grille cible est déjà gagnée ou complète, le joueur suivant est libre de choisir n'importe quelle grille jouable (`isBoardForced` passe à `false`).
- Une petite grille gagnée ne peut plus recevoir de coups (`isWon == true`).
- Le vainqueur est le joueur qui aligne 3 petites grilles remportées horizontalement, verticalement ou en diagonale.
- En cas d'égalité (aucun alignement possible), le joueur ayant remporté le plus de petites grilles gagne — gestion prévue par `isGameOver()` combiné à `getWinner()`.